

Non-Interactive Distributed Key Generation on the EVM with Per-Application Committee Keys

Anonymous Submission

Abstract. We present a distributed key generation protocol on a public EVM chain in which every contribution is a publicly verifiable Feldman dealing carried by a Groth16 proof and verified with one fixed-size check over eight public inputs, whatever the committee size, so no complaint phase exists. One epoch deals a pool of $K_{\max}$ independent committee keys inside the same proof, each activated by a small separate proof. Every application claims its own pool key, so a copied ciphertext decrypts under an unrelated key, or adds an organizer key revealed once, application-wide, before which the contract admits no partial decryption. A secrecy game with corrupt organizers, submitters and f committee members admits DDH reductions for both modes under stated corruption, proof-system and random-oracle assumptions. At a committee bound of 32 and a pool of 8, the contribution circuit has 3.06 M constraints and proves in about 1.6 s; a contribution transaction costs 0.70–0.94 M gas.

Keywords: Distributed key generation · Threshold decryption · Publicly verifiable secret sharing · zk-SNARKs · EVM.

1 Introduction

A distributed key generation protocol lets n parties produce a public key whose secret is Shamir-shared so that any t of them can decrypt [17], while nobody ever holds it. Pedersen-style DKGs [35] let recipients check their own shares and settle faulty dealings by interactive complaints; on a public chain that dispute logic runs in a contract, at block granularity, and is itself an attack surface [36,42]. Publicly verifiable secret sharing [43,37,21] removes complaints in principle, but its proofs cost $\Theta(n)$ group operations per dealing, in groups the EVM does not expose or prices at hundreds of thousands of gas, and the contract must verify every dealing of every epoch.

A general-purpose succinct proof changes the accounting: a Feldman dealing carried by a Groth16 proof is accepted or rejected by one fixed-size verification with eight public inputs, whatever n and t ; what still grows with the committee is the calldata of the encrypted shares, so a contribution transaction costs 0.70 M gas at $n = 4$ and 0.94 M at $n = 32$ in our build (§7). The rest of the paper makes that check bind the right data and lets one committee serve many applications under keys that do not leak into each other.

Motivating application. Consider an encrypted voting service: ballots are exponential-ElGamal ciphertexts, aggregated homomorphically off chain, and only authorized tally ciphertexts are submitted for threshold decryption. The aggregate submitter does not know the summed randomness, so the proof of knowledge with which threshold ElGamal classically closes its decryption oracle [41] is unavailable. Elections need isolated keys; some also need an organizer-controlled, application-wide release. Ballot validity, eligibility, aggregation correctness and tally authorization lie outside this layer, and the protocol alone does not stop a malicious authorized submitter from submitting an individual ballot.

Contributions.

1. *Succinctly verified dealings of a key pool.* Each contribution deals $K_{\max}$ independent Feldman polynomials to the whole committee and is verified on chain by one fixed-size Groth16 verification with eight public inputs; an invalid dealing is refused at submission, so no dispute phase exists. Finalization freezes the accepted dealers without a proof; each pool key is activated on demand by a small proof that rebinds, through a two-level digest, commitments already published (§4).
2. *Per-application keys in two modes, with a security model.* Every application claims one pool key. An *automatic* application is decrypted by any t members inside its decryption window; an *organizer-locked* one adds an organizer key, $PK_{\text{aid}} = P_j + PK_{\text{org}}$, whose secret is revealed once, application-wide; with fewer than t corrupt members, honest nodes release no partial before that gate opens. A secrecy game with corrupt organizers, submitters and f members yields DDH reductions for a locked application before its reveal ($n - f \geq t$, Theorem 2) and for an automatic one ($f < t$, Theorem 1), with their assumptions stated (§6).
3. *Verification devices and an implementation.* A binding random-linear-combination commitment keeps every verifier at 8–15 public inputs; we state its exact hypotheses and the forgery when one is dropped (Lemma 1). A point-based Vandermonde check pins the combine proof’s interpolation vector (§5). Three contracts and four circuits are measured with their real verifiers on a local testbed (§7).

2 Related Work and Comparison

Pedersen’s DKG [35] runs n Feldman instances [19] in parallel; Gennaro et al. showed that a rushing adversary biases its key [23] and that the biased key stays safe for discrete-log applications such as ElGamal [24], the position this paper takes. Kate and Goldberg moved DKG to the Internet setting [31]; asynchronous DKGs [33,16,3,15,46,2,20] reach near-quadratic communication and adaptive security but rest on asynchronous VSS and agreement primitives [4], unneeded on a chain that already orders messages; SPRINT [7] targets threshold signing.

Complaint-free dealing is not new. Publicly verifiable secret sharing [43,37] and Fouque and Stern’s one-round DKG [21] began the line; SCRAPE [11] made

Table 1. Dealing verification in DKGs with a public bulletin board. PV: publicly verifiable dealing. SRS: per-circuit trusted setup (the other rows need none). Size: what one dealing publishes.

Protocol	Check	Verifier per dealing	Size per dealing	Verified by
Pedersen; Gennaro et al.; Shutter [35,23,44]	complaints	$O(t)$ exp., recipient only	$O(t) + n$ private	parties; synchronous rounds
Fouque-Stern; SCRAPE; ALBATROSS; Kate et al. [21,11,12,32]	PV	$O(n)$ Paillier, group or class-group ops	$O(n)$	anyone
Gurkan et al.; Groth NI-DKG [28,27]	PV	$O(n)$ exp. + pairings; aggregatable	$O(n)$	anyone; BFT replicas
Ferveo [5]	PV	$O(n)$ exp. + pairings	$O(n)$	BFT validators
EthDKG [36]	complaints on chain	$O(t)$ exp. per recipient; contract adjudicates	$O(t + n)$	EVM contract; block windows
Sober et al. [42]	complaints on chain	SNARK-checked rebuttal (SRS)	$O(t + n)$	EVM contract; block windows
This work	PV	one Groth16 verification, 8 public inputs (SRS)	928 words, $K_{\max}$ dealings	EVM contract; block windows

verification linear, ALBATROSS [12] batched it, Gurkan et al. [28] made transcripts aggregatable, Groth’s NI-DKG [27] uses forward-secure encryption and a bespoke proof system, Kate et al. [32] need no setup at all with class groups, and Ferveo [5] deploys publicly verifiable dealings among the validators of a BFT chain. Each verifier performs $\Theta(n)$ group operations per dealing.

On the EVM, EthDKG [36] ports Joint-Feldman with on-chain dispute resolution, Sober et al. [42] check the rebuttal with a zk-SNARK but keep the complaint phase, and Shutter [44] runs Feldman VSS with interactive complaints in production. Table 1 positions the present design: a publicly verifiable dealing, checked by the contract at a fixed verifier cost, under a per-circuit trusted setup. It separates verifier work from transcript size and infers no full transaction cost from asymptotics.

3 Model, Goals and Notation

3.1 Notation and Primitives

$\mathcal{G}$ is a cyclic group of prime order q , written additively with generator G ; DDH is assumed hard. The implementation uses BabyJubJub’s prime-order subgroup (cofactor 8) over the BN254 scalar field $\mathbb{F}_p$, $p > q$, the circuits’ native field, so in-circuit reduction modulo q needs a bounded quotient (§5). H_K is Keccak-256, reduced modulo q for the contract’s Schnorr challenges and modulo p for the calldata-binding challenges of §5; H_P is Poseidon [25]. The contract checks points canonical, on-curve and, where meaningful, non-identity; whoever multiplies a secret into a point checks its subgroup. An epoch has $n \leq N_{\max}$ members, threshold t , f corrupt, and $K_{\max}$ pool keys ($N_{\max} = 32$, $K_{\max} = 8$); $QUAL \subseteq$

$[n]$, $|\text{QUAL}| \geq m_{\min} \geq t$, are the accepted dealers, frozen at finalization; $\mathcal{Q}$, $|\mathcal{Q}| = s \geq t$, those whose partials are combined.

Feldman VSS [39,19]. A dealer with secret σ samples a uniform f of degree below t over $\mathbb{F}_q$ with $f(0) = \sigma$, sends party u the share $f(u)$ and publishes $C(k) := a_k G$ per coefficient; u checks

$$f(u)G \stackrel{?}{=} \sum_{k=0}^{t-1} u^k C(k). \quad (1)$$

Hashed ElGamal for shares. A share $s \in \mathbb{F}_q$ is encrypted under a recipient key $\text{pub} = xG$ as

$$(R, \sigma) := (rG, s + \text{Hp}(\text{ctx} \parallel r \text{ pub}) \bmod q), \quad r \leftarrow \mathbb{F}_q, \quad (2)$$

and recovered as $s = \sigma - \text{Hp}(\text{ctx} \parallel xR)$; ctx binds epoch, dealer, recipient and key index, so one ECDH value per recipient yields $K_{\max}$ distinct masks; IND-CPA holds under hash-CDH [1,13]. Applications use exponential ElGamal (§4).

Proofs. Groth16 [26] needs a per-circuit structured reference string and is knowledge-sound only in the generic-group model, which every dependent statement inherits.

3.2 Chain, Parties and Adversary

Chain and parties. We assume finality below the parties' confirmation depth, bounded inclusion of funded honest parties' transactions (phase windows are sized accordingly) and availability of accepted calldata. Registered operators form each epoch's committee by snapshot-based eligibility filtering from a block-hash seed (§4), exposed to proposer bias and inclusion races: every result is conditioned on the selected committee meeting the corruption bound. Organizers register applications and submitters post ciphertexts; registry admission is a deployment assumption.

Adversary. Corruptions are static [9]: f members and any number of other operators, organizers and submitters. $n - f \geq t$ gives availability and lets the challenger of §6 interpolate every pool secret without rewinding; $f < t$ plus an honest accepted dealer ($m_{\min} > f$) is needed wherever a pool secret must stay hidden; honest members keep shares for the epoch's decryption horizon. We assume (a) DDH in $\mathcal{G}$; (b) H_K a random oracle (Theorem 2, Lemma 1); (c) H_P collision resistant, a random oracle as in-circuit challenge; (d) knowledge soundness for adversarial dealings (Lemma 2), zero knowledge where Theorem 1 simulates honest proofs; (e) IND-CPA of (2). Adaptive or proactive corruption, UC composition and setup compromise are out of scope.

Table 2. Guarantees by adversary. Outsiders: other organizers and submitters. Gated: honest members post no partial before the application’s window opens and, if locked, before its reveal. Row 1 also needs one honest accepted dealer.

Adversary controls	Bound	What holds
Outsiders; f members	$f < t$	Honest application confidential while gated (Theorem 1)
Own organizer; f members	$f < t$	No leak via other applications; own one gated (§6)
Outsiders; f members	$n - f \geq t$	Locked application confidential until reveal (Theorem 2)
t members	—	Automatic mode broken; locked keeps its organizer factor
Organizer and t members	—	No embargo in either mode; coalition decrypts off chain
Authorized submitter	any	Chooses what enters; nothing is claimed about its inputs

3.3 Goals

Correctness: for an activated key j , $A_j := \sum_{\ell \in \text{QUAL}} f_\ell^{(j)}$, the stored key is $A_j(0)G$, the Merkle leaves $A_j(i)G$, honest member i ’s share $A_j(i)$, and a combined plaintext of a well-formed ciphertext $\log_G(C_2 - \text{sk}_{\text{aid}}C_1)$ within the search bound. *Availability:* an epoch with $m_{\min}$ accepted contributions goes live with every key activatable; a well-formed ciphertext whose gate is open is decrypted whenever t honest holders are available and included. *Secrecy:* the game of §6; Table 2 states what holds under which corruption, the gate bounding only what the contract accepts and honest nodes publish, never t colluding share holders.

4 Protocol and Design Rationale

The protocol runs in *epochs*. An epoch **eid** deals a pool of $K_{\max}$ independent keys $P_0, \dots, P_{K_{\max}-1}$, contributed by the members $\text{QUAL} \subseteq [n]$ whose dealings were accepted; each application registered against it claims one key. Operators hold registered share-encryption keys pub_i (§7); admission is a deployment concern. On the block axis an epoch passes a selection window, a contribution window and a key-assembly gap, then stays **LIVE** with keys activated on demand.

4.1 Committee Selection

The initiator submits $(t, n, m_{\min}, \alpha)$ —threshold, committee size, minimum accepted contributions, oversubscription—and the contract snapshots the number N of active operators, pins a seed block b_s a few blocks ahead and stores $\tau := \lfloor 2^{256} \alpha n / N \rfloor$. The first claim after b_s freezes $\text{seed} := \text{blockhash}(b_s)$; operator **op** is admissible iff $\text{keccak}(\text{seed} \parallel \text{op}) < \tau$, and the first n admissible claimants take the slots. Only operators active before creation may claim, so the seed cannot be ground with fresh identities. This is snapshot-based *eligibility filtering*, not an unbiased sampler: the seed’s block proposer can resample or withhold it, and slots go first come, first served, so inclusion latency decides among admissible claimants. §6 assumes the selected committee meets the corruption bound.

Epochs are created permissionlessly once **EPOCH_DURATION** blocks have elapsed, or earlier once the newest epoch is **LIVE** with at most one unclaimed key or aborted. Deployment immutables floor t and n and cap α ; whoever wins the

creation race can pin every epoch to them. Robustness depends on the chosen epoch's $n - t$: the floors enforce no margin ($t = n$ is admissible), and tolerating r unavailable members needs $t \leq n - r$.

4.2 Epoch Key Generation

Contribution. Each member i samples, for every key $j \in [K_{\max}]$, a uniform polynomial $f_i^{(j)}(x) := \sum_{k < t} a_{i,j,k} x^k$ over $\mathbb{F}_q$ and one nonce $r_{i,u}$ per recipient $u \in [n]$, and publishes in one transaction the commitments $C_i^{(j)}(k) := a_{i,j,k} G$ and, for every (j, u) , the encrypted share

$$E_i^{(j)}(u) := (R_i(u), \sigma_i^{(j)}(u)) = (r_{i,u}G, f_i^{(j)}(u) + H_P(\text{ctx}_{i,u} \| j \| r_{i,u} \text{pub}_u) \bmod q)$$

and a proof π_i that (1) holds and every $E_i^{(j)}(u)$ encrypts $f_i^{(j)}(u)$ (§5). The commitments are digested per key, and the $K_{\max}$ digests again into a single **commitmentsHash**, which the contract stores after verifying π_i ; the shares stay in calldata. By Lemma 2 an accepted contribution is a correct dealing to all n recipients, so no complaint phase [35,23] is needed.

Finalization. After the contribution window, with at least $m_{\min}$ accepted contributions and the key-assembly gap elapsed, anyone may call **finalizeEpoch**: it takes *no proof*, freezes **QUAL** as exactly the accepted set and marks the epoch **LIVE**. An epoch that missed its committee or its $m_{\min}$ rows is dead; finalization has no deadline.

Pool-key activation. Let $A_j := \sum_{\ell \in \text{QUAL}} f_\ell^{(j)}$, $\overline{C}^{(j)}(k) := \sum_{\ell \in \text{QUAL}} C_\ell^{(j)}(k)$, $P_j := \overline{C}^{(j)}(0) = A_j(0)G$ and $D_{j,i} := \sum_{k < t} i^k \overline{C}^{(j)}(k) = A_j(i)G$ for $i \in [n]$. While the epoch is **LIVE** anyone may activate any key j by a proof that the $\overline{C}^{(j)}(k)$ aggregate exactly the accepted key- j commitments behind the stored digests (§5) and that the $D_{j,i}$ are their evaluations at every committee position. The contract stores P_j and root_j , a Keccak Merkle root over $N_{\max}$ tagged leaves $\text{keccak}(0x00 \| D_{j,i})$ for $i \leq n$, the constant $\text{keccak}(\text{dom}_{\text{empty}})$ beyond n , and inner nodes $\text{keccak}(0x01 \| l \| r)$. Every member $i \in [n]$ obtains $e_{j,i} := A_j(i)$ by decrypting the calldata, whether or not it dealt: decryption tolerates $n - t$ absent members.

4.3 Applications and Their Keys

Registration against a **LIVE** epoch names a non-zero field element **aid** and a policy, and claims the next activated, unclaimed key P_j . In the *organizer-locked* mode the registrant, the organizer, also submits $\text{PK}_{\text{org}} = \text{sk}_{\text{org}}G$ with a Schnorr proof of possession $(A, z) = (wG, w + c \text{sk}_{\text{org}})$, where $c := H_K(\text{dom}_{\text{org}} \| \text{eid} \| \text{aid} \| \text{PK}_{\text{org}} \| A)$, checked as $zG = A + c \text{PK}_{\text{org}}$; it excludes rogue keys such as $-P_j$. The application key is $\text{PK}_{\text{aid}} := P_j + \text{PK}_{\text{org}}$. In the *automatic* mode there is no

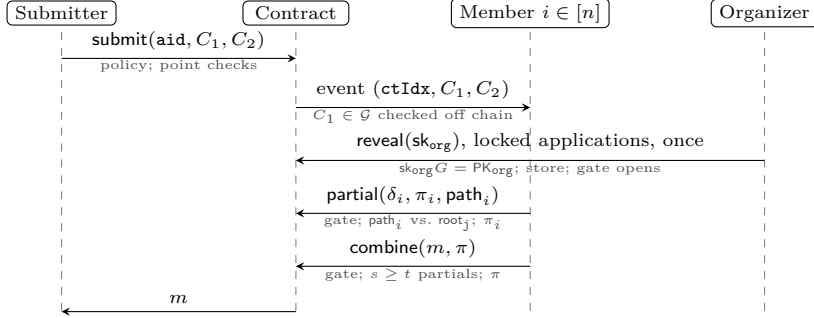


Fig. 1. Decryption of a ciphertext on pool key P_j ; arrows into the contract are transactions. Partials and the combine carry Groth16 proofs, partials also a Merkle path for $D_{j,i}$; the gate is the decryption window plus, for locked applications, the one-shot reveal.

organizer key: the contract records the identity and 0, $\text{PK}_{\text{aid}} := P_j$, and any t share holders decrypt.

A user encrypts m as $(C_1, C_2) := (rG, mG + r\text{PK}_{\text{aid}})$, $r \leftarrow \mathbb{F}_q$: exponential ElGamal, additively homomorphic, m being recovered from mG by a bounded search. Because each key is dealt by independent polynomials, the committee's term $A_j(0)C_1$ is specific to the application that claimed P_j : a C_1 copied into a decoy application is answered with $A_{j'}(0)C_1$ under an unrelated key; Theorems 1 and 2 make this precise.

Policy. Registration fixes who may submit (anyone, an allow-list or the registrant), a submission block window and a decryption window. Ciphertexts are accepted only inside the submission window and before decryption closes; partials and combines only inside the decryption window and, for a locked application, after the reveal. Window and reveal form one application-wide gate on what the contract accepts: with fewer than t corrupt members, honest nodes release no partial before it opens; it does not bind t colluding members (§8).

4.4 Threshold Decryption

Submission. `submitCiphertext(aid, C1, C2)` carries no proof (Figure 1): the contract enforces the policy and a per-application cap, assigns `ctIdx` on chain and checks both points well formed and not the identity. It does not check that C_1 lies in the prime-order subgroup; a cofactor component would leak $e_{j,i} \bmod 8$ through a partial, so the member about to multiply its share into C_1 checks this off chain.

Partial decryption. Any member $i \in [n]$ publishes $\delta_i := e_{j,i}C_1$ with a DLEQ proof [14] $(A_i, B_i, z_i) := (w_iG, w_iC_1, w_i + c_i e_{j,i})$, $w_i \leftarrow \mathbb{F}_q$, with challenge $c_i := \text{Hp}(\text{dom} \parallel \text{eid} \parallel \text{aid} \parallel \text{ctIdx} \parallel i \parallel D_{j,i} \parallel C_1 \parallel \delta_i \parallel A_i \parallel B_i)$, verified as $z_iG = A_i + c_i D_{j,i}$ and

$z_i C_1 = B_i + c_i \delta_i$ inside a Groth16 proof (§5), and a Merkle path for $D_{j,i}$ that the contract checks against root_j at leaf $i - 1$.

Reveal. A locked application’s organizer publishes sk_{org} through the permissionless, one-shot `revealOrganizerSecret` call; the contract checks $\text{sk}_{\text{org}} G = \text{PK}_{\text{org}}$ and stores it. Until then no partial or combine of the application exists on chain; afterwards every past and future ciphertext of the application is eligible under the same gate. The organizer decides *when*, not *which* (§8).

Combine. Once $s \geq t$ partials from $\mathcal{Q} = \{x_1, \dots, x_s\}$ are on chain and the gate is open, anyone submits a proof establishing, with the canonical Lagrange coefficients λ_k of $\mathcal{Q}$ (§5),

$$\sum_k \lambda_k \delta_{x_k} = A_j(0) C_1, \quad M = C_2 - \sum_k \lambda_k \delta_{x_k} - \text{sk}_{\text{org}} C_1 = mG, \quad (3)$$

and, from the private witness sk_{org} , that $\text{sk}_{\text{org}} G$ is the PK_{org} on record, trivially for an automatic application. The combiner recovers m off chain by baby-step giant-step [40] (§7). With fewer than t share holders left, the epoch’s ciphertexts are undecryptable (§8).

Design rationale. The design is constrained by clients that already produce exponential ElGamal ciphertexts and homomorphic aggregates, an organizer that runs no prover, and a DKG that verifies nothing application-specific. A proof of knowledge of r with $C_1 = rG$ [41], the classical way to tie a ciphertext to its submitter, is unavailable: the submitter of an aggregate $\sum_k C^{(k)}$ does not know $\sum_k r_k$. The committee therefore answers any C_1 under any key it serves, so separation cannot come from a public additive offset— $\text{PK}_{\text{ep}} + \mathcal{H}(\text{aid}) G$ [45] is removed by shifting C_2 by $\mathcal{H}(\text{aid}) C_1$ —but only from independently dealt keys. Dealing them as a pool inside the epoch’s contribution costs one transaction and one proof per member for K_{max} keys; the pool is finite and refilled by the next epoch.

5 Succinct Verification

Recurring operations are proven in Groth16 circuits [26] over BN254 with BabyJubJub embedded and Poseidon [25] as hash. Circuits have fixed capacity $N_{\text{max}} = 32$, $K_{\text{max}} = 8$ (inactive slots masked to constants), check points on the curve, range-check scalars used modulo q and use hint-free variable-base multiplication [30]. Contribution $\mathcal{R}_{\text{deal}}$ has public inputs $(\text{eid}, t, n, i, h_C, h_E, \rho, \beta)$; activation $\mathcal{R}_{\text{act}}$ has $(\text{eid}, t, n, |\text{QUAL}|, j, d, \rho, \beta)$ with the transcript digest d at index 5 of 8; partial decryption $\mathcal{R}_{\text{part}}$ has the 15 scalars $(\text{eid}, \text{aid}, \text{ctIdx}, i, C_1, D_{j,i}, \delta_i, A_i, B_i, z_i)$, c_i as in §4; combine $\mathcal{R}_{\text{comb}}$ has $(\text{eid}, \text{aid}, \text{ctIdx}, t, s, h, m, \rho, \beta)$, $s = |\mathcal{Q}|$. Indices range over $j < K_{\text{max}}$, $u, i \in [n]$, $\ell \in \text{QUAL}$, $k < t$ ($k \leq s$ in $\mathcal{R}_{\text{comb}}$); all else is witness, $\mathbf{w}$ the transcript committed

by $\beta = \text{BRLC}_\rho(\mathbf{w})$ and $h_C \parallel h_E, d, h$ its Poseidon digests (§5.1); $g_{\ell,j'}$ is defined below:

$$\begin{aligned}
\mathcal{R}_{\text{deal}} : & a_{i,j,k} \in [0, q), \quad C_i^{(j)}(k) = a_{i,j,k} G, \quad R_i(u) = r_{i,u} G, \quad S_{i,u} = r_{i,u} \text{pub}_u, \\
& s_i^{(j)}(u) \in [0, q), \quad s_i^{(j)}(u) G = \sum_{k < t} u^k C_i^{(j)}(k), \quad h_E = \text{Hp}(\text{pub} \parallel \mathbf{E}_i), \\
& \sigma_i^{(j)}(u) = s_i^{(j)}(u) + \text{Hp}(\text{ctx}_{i,u} \parallel j \parallel S_{i,u}) \bmod q, \quad \mathbf{w} = \mathbf{C}_i \parallel \text{pub} \parallel \mathbf{E}_i, \\
& h_C = \text{Hp}(\text{eid}, i, t, \text{Hp}(\mathbf{C}_i^{(0)}), \dots, \text{Hp}(\mathbf{C}_i^{(K_{\max}-1)})); \\
\mathcal{R}_{\text{act}} : & t \leq |\text{QUAL}| \leq n \leq N_{\max}, \quad j < K_{\max}, \quad h_{C,\ell} = \text{Hp}(\text{eid}, \ell, t, g_{\ell,0}, \dots, g_{\ell,K_{\max}-1}), \\
& \overline{C}^{(j)}(k) = \sum_{\ell \in \text{QUAL}} C_\ell^{(j)}(k), \quad D_{j,i} = \sum_{k < t} i^k \overline{C}^{(j)}(k), \\
& \mathbf{w} = \text{QUAL} \parallel \{h_{C,\ell}\} \parallel \overline{C}^{(j)} \parallel \mathbf{D}_j \in \mathbb{F}_p^{6N_{\max}}, \quad d = \text{Hp}(\text{eid}, j, \mathbf{w}); \\
\mathcal{R}_{\text{part}} : & (D_{j,i}, \delta_i) = e_{j,i}(G, C_1), \quad (A_i, B_i) = w_i(G, C_1), \\
& z_i(G, C_1) = (A_i, B_i) + c_i(D_{j,i}, \delta_i); \\
\mathcal{R}_{\text{comb}} : & t \leq s, \quad m, \text{sk}_{\text{org}} \in [0, q), \quad \text{PK}_{\text{org}} = \text{sk}_{\text{org}} G, \quad \sum_{k \leq s} \lambda_k x_k^r G = [r=0] G \quad (r < s), \\
& C_2 = mG + \sum_{k \leq s} \lambda_k \delta_{x_k} + \text{sk}_{\text{org}} C_1, \quad h = \text{Hp}(\text{eid}, \text{aid}, \text{ctIdx}, t, s, \mathbf{w}), \\
& \mathbf{w} = C_1 \parallel C_2 \parallel \text{PK}_{\text{org}} \parallel \{x_k\} \parallel \{\delta_{x_k}\} \in \mathbb{F}_p^{6+3N_{\max}}.
\end{aligned}$$

The range checks and the bounded-quotient mask reduction $\text{Hp}(\cdot) = \text{quot} \cdot q + \text{mask}$ ($\text{quot} \leq 7$, $\text{mask} < q$, and $\text{mask} < p - 7q$ if $\text{quot} = 7$) make every share and mask canonical over the integers. In $\mathcal{R}_{\text{act}}$ the contributors' commitments are witness bound by the stored $h_{C,\ell}$, so the transcript has $6N_{\max}$ words; the contract reads $P_j = \overline{C}^{(j)}(0)$ and $\mathbf{D}_j$ from the commitment-bound calldata and stores P_j and root_j (§4). $\mathcal{R}_{\text{part}}$ keeps (A_i, B_i, z_i) a self-contained DLEQ proof [14]; $\mathcal{R}_{\text{comb}}$ proves knowledge of the sk_{org} read from the application record (0 for an automatic application). Before the pairing check the contract also ties inputs to state (QUAL rows and $h_{C,\ell}$, the digest at input 5, C_1 , the Merkle path of $D_{j,i}$ against root_j , PK_{org} , the stored $\text{keccak}(\delta_{x_k})$) and gates both decryption calls (§4).

Canonical interpolation. Since $\mathbb{F}_p \neq \mathbb{F}_q$ the λ_k cannot be computed by division; $\mathcal{R}_{\text{comb}}$ checks the square Vandermonde system on points, where arithmetic is modulo q . For s distinct non-zero nodes, which the contract enforces, its unique solution is the canonical Lagrange vector at 0 of $\mathcal{Q}$. Correctness and uniqueness differ: the first t rows already make $\sum_k \lambda_k \delta_{x_k}$ interpolate the shared polynomial of degree below t , but for $s > t$ they admit an $(s - t)$ -dimensional family of such vectors, which the remaining rows exclude; without them a prover knowing $\log_G \delta_{x_k}$ for one k —the encryptor colluding with one member—could shift λ_k and open the ciphertext to any plaintext (§6).

Two-level digests. The contract keeps one Poseidon word $h_{C,\ell}$ per contribution and nothing per key, yet activating key j must show that P_j aggregates exactly the accepted key- j commitments without posting or hashing every contributor's $2K_{\max}N_{\max}$ -word matrix. The contribution digest is therefore nested, $h_{C,\ell} = \text{Hp}(\text{eid}, \ell, t, g_{\ell,0}, \dots, g_{\ell,K_{\max}-1})$ with $g_{\ell,j} = \text{Hp}(\mathbf{C}_\ell^{(j)})$, and $\mathcal{R}_{\text{act}}$ takes per contributor the $2N_{\max}$ words of key j and the $K_{\max} - 1$ other digests as witness, recomputes $g_{\ell,j}$ and reproduces the stored word, $2N_{\max} + K_{\max}$ absorptions instead of $2K_{\max}N_{\max}$. By collision resistance at both levels no other contribu-

tor's or key's commitments can be substituted, so each key is rebound separately without republishing the others.

5.1 Binding Random Linear Combinations

Passed naively, $\mathcal{R}_{\text{deal}}$ would have $2K_{\max}t + (3 + K_{\max})n$ public scalars, over 2.5 M gas at $n = 20$, $t = 10$; instead each circuit exposes a digest d of its witness vector $\mathbf{w} \in \mathbb{F}_p^l$ and one word $\beta = \text{BRLC}_\rho(\mathbf{w}) := \sum_{k=1}^l \rho^k w_k \bmod p$, which the contract recomputes over the l calldata words $\mathbf{v}$ (Freivalds [22]). It binds only if ρ is independent of *both* vectors: were ρ derived from the prover's digests alone, the prover would know it before choosing the calldata and could satisfy one linear equation over $\mathbb{F}_p$ —change v_a by η and v_b by $-\eta\rho^{a-b}$ —freeing the words the contract reads straight from calldata, P_j among them; a witness word no digest fixes fails dually. The challenge therefore covers both sides, $\rho := \text{keccak}(\text{eid} \parallel \text{dom} \parallel \text{keccak}(d \parallel \text{keccak}(\mathbf{v}))) \bmod p$, where d absorbs every witness word the circuit commits to, inactive slots masked identically throughout, and dom is a per-relation constant; the anchors are $h_C \parallel h_E$, d and $h \parallel m$ respectively.

Lemma 1 (BRLC binding). *Let $\mathbf{v} \neq \mathbf{w} \in \mathbb{F}_p^l$ be fixed and $\rho = x \bmod p$ for uniform $x \in [0, 2^{256})$; then $\Pr[\text{BRLC}_\rho(\mathbf{v}) = \text{BRLC}_\rho(\mathbf{w})] \leq \gamma l/p$, $\gamma = \lceil 2^{256}/p \rceil p/2^{256} < 1.135$. Hence, with keccak a random oracle [6], a Q -query prover that has the contract accept l canonical calldata words $\mathbf{v}$ and a proof with public inputs $d = \text{H}_P(\mathbf{w})$, ρ as above and $\text{BRLC}_\rho(\mathbf{w})$, $\mathbf{w} \neq \mathbf{v}$, succeeds with probability at most $Q\gamma l/p$ plus its advantage against H_P 's collision resistance and the proof system's knowledge soundness.*

Proof. The polynomial $\sum_{k=1}^l (w_k - v_k)X^k$ is non-zero of degree at most l , so it has at most l roots [38], each hit with probability at most $\lceil 2^{256}/p \rceil / 2^{256} = \gamma/p$. The accepted ρ answers a query $\text{keccak}(\text{eid} \parallel \text{dom} \parallel \text{keccak}(d \parallel h))$ in which h fixes $\mathbf{v}$ and d fixes the extracted $\mathbf{w}$ unless a hash collides, so both are determined before the answer is drawn; a union bound over Q queries finishes. $\square$

By construction both vectors have the circuit's fixed length l , the contract rejects words $\geq p$, so a transcript has one encoding, and the query encoding is fixed-width; only the root bound is statistical, below $2^{-243.5}$ at $l = 928$ against $2^{-243.7}$ for a uniform challenge.

6 Security analysis

Model. We consider one epoch of §4, conditional on its selected committee. A static adversary controls f members, arbitrary other organizers and submitters, and transaction scheduling within the chain assumptions; it sees the public transcript and may rush honest dealings. Honest nodes validate subgroup membership before releasing partials. Availability further requires finalized, available calldata, eventual transaction inclusion, funded participation, and

retained shares. We establish neither committee-capture resistance nor adaptive-corruption or universal-composition security. Secrecy concerns an honestly encrypted challenge with fresh randomness, not plaintext provenance, ballot validity, or secrecy after authorized release.

We use the following assumptions. (a) DDH is hard in the prime-order group $\mathcal{G} = \langle G \rangle$. (b) H_K is a 256-bit random oracle; possession-proof challenges reduce its output modulo q , BRLC challenges modulo p . (c) H_P is collision resistant for transcript binding; its share-mask invocations are additionally idealized as a random oracle over $\mathbb{F}_p$ [6]; collision resistance alone does not imply mask secrecy. (d) Independently generated circuit reference strings provide auxiliary-input knowledge soundness and multi-theorem zero knowledge; the instantiated SNARK’s soundness analysis inherits its generic-group idealization [26], and compromised setup is excluded. (e) Honest parties use independent randomness and authenticated keys; pool-key secrecy requires $f < t$, $n - f \geq t$, and an honest dealer in QUAL—properties of the selected epoch, not guarantees of parameter floors. (f) Contribution proofs are additionally auxiliary-input, online, multi-theorem *simulation-extractable*: after simulated proofs, including proofs of false statements, witnesses remain extractable from fresh accepting adversarial statements in the joint setup and oracle environment; this stronger composition property is not established here for the instantiated SNARK.

Let ϵ_{ks} denote aggregate knowledge-soundness/extraction failure, and ϵ_{bind} the failure bound of Lemma 1—with all its encoding, digest-coverage, and context hypotheses, and using the actual modulo- p challenge distribution rather than an ideal uniform challenge—plus remaining digest/Merkle collisions. Write $\epsilon_0 = \epsilon_{ks} + \epsilon_{bind} + \epsilon_{rng}$ with $\epsilon_{rng} \leq N_{sc}/q$, the cost of idealizing N_{sc} prescribed nonzero scalar samples as uniform over $\mathbb{F}_q$.

Lemma 2 (Dealing correctness). *Except with probability $\epsilon_{ks} + \epsilon_{bind}$, every accepted contribution encrypts evaluations of degree-below- t polynomials over $\mathbb{F}_q$, consistent with its coefficient commitments; its stored digest computationally binds all K_{max} commitment vectors.*

Proof. Soundness and Lemma 1 bind a relation witness to canonical calldata; coefficient commitments and share equations identify the polynomial evaluations, and Appendix A.3 establishes canonical masking arithmetic. Both parties compute the same ECDH point, so recipient subtraction recovers those evaluations.

Consequently, activation binds $P_j = A_j(0)G$ and every committee member’s $D_{j,i} = A_j(i)G$ under root_j , where $A_j = \sum_{d \in \text{QUAL}} f_d^{(j)}$, and root-checked partials satisfy $\delta_i = A_j(i)C_1$. The implemented combine relation in §5 checks $\sum_{k=1}^s \lambda_k x_k^\ell G = [\ell = 0]G$ for $0 \leq \ell < s$, $s = |\mathcal{Q}| \geq t$, hence uses the canonical interpolation vector; checking only t rows would ensure interpolation correctness but not uniqueness when $s > t$. With the organizer-secret relation this proves the recovered message point correct.

Secrecy experiment. The adversary may register applications, submit arbitrary ciphertexts, and obtain every policy-admitted reveal and honest partial on other

applications, including for copied challenge components. For pool-key secrecy, the target has never released an honest partial and the experiment ends before its gate first opens. The challenge encrypts one of two chosen messages; advantage is $|\Pr[\hat{b} = b] - 1/2|$, and inadmissible or aborted executions return a random guess, so bounds are not renormalized by successful epoch completion. There is no target-key decryption oracle and no same-application CCA claim.

Theorem 1 (Automatic-application secrecy and isolation, conditional). *Under assumptions (a)–(f), the experiment for an automatic application under its uniquely claimed pool key satisfies $\text{Adv}_{\text{auto}}(\mathcal{A}) \leq L \text{Adv}_G^{\text{ddh}}(\mathcal{B}) + R\epsilon_{\text{HE}} + \epsilon_{\text{zk}} + \epsilon_{\text{se}} + \epsilon_0$. Here $L \leq K_{\text{max}}(n - f)$ accounts for guessing the target slot and an honest accepted dealer, $R \leq (n - f)^2$ counts honest-dealer/honest-recipient rows, ϵ_{HE} is the per-row hashed-encryption hybrid loss, and $\epsilon_{\text{zk}}, \epsilon_{\text{se}}$ are whole-experiment simulation and simulation-extractability losses.*

Proof. Proof sketch. In $H_0 \rightarrow H_1$, replace honest contribution proofs on true statements by simulated proofs. In $H_1 \rightarrow H_2$, replace each honest-recipient row’s K_{max} masked shares by independent uniform placeholders, recomputing its digest and simulated proof; one ECDH secret serves the row, but the key index separates its hash inputs, and the transition includes encryption security and reduction bias, not merely zero knowledge. Honest private states are maintained virtually. The modified dealing statements may be false: assumption (f), rather than ordinary zero knowledge, justifies applying dealing correctness to subsequent adversarial contributions, and during the encryption reductions its extractor also supplies adversarial polynomials when an embedded recipient’s decryption secret is unavailable. In H_2 , embed $X = uG$ in one honest dealer’s target-key constant commitment. Interpolation in the exponent supplies coefficient commitments while fixing the $f < t$ corrupt recipients’ evaluations to known scalars; honest-recipient target evaluations need not be known because their ciphertexts are placeholders, and activation still has a genuine point-valued witness. In the final reduction all recipient encryption secrets are known: decrypt each adversarial dealing at t honest positions (using $n - f \geq t$) and interpolate its polynomial. Including other honest dealers gives a known, possibly adaptively chosen offset a with $P_j = X + aG$, and every other key’s shares are known, so its oracles remain answerable. For a DDH tuple (G, X, Y, Z) , use $(C_1^*, C_2^*) = (Y, m_b G + aY + Z)$: the DH case is the challenge encryption, and independent uniform Z hides b . Appendix A.2 supplies the bookkeeping. This proves computational isolation, not uniformity of adversarially biased output keys [23].

Theorem 2 (Locked-application secrecy). *Under (a)–(d) and honest randomness, a locked target with an independent, unrevealed organizer secret remains secret even after disclosure of every pool share, including against $f \geq t$ members without that secret: $\text{Adv}_{\text{locked}}(\mathcal{A}) \leq Q_A(\text{Adv}_G^{\text{ddh}}(\mathcal{B}) + Q_K/q) + \epsilon_0$, where Q_A bounds honest organizer registrations and Q_K counts H_K queries. Conversely, under (a)–(f), giving the adversary the target’s correctly generated organizer secret preserves Theorem 1’s bound while the target gate remains closed.*

Proof. Proof sketch. Recover the pool polynomials from t honest recipients; if $n - f < t$, use (d)’s contribution extractor before any organizer-proof simulation. Embed $X = uG$ as the organizer key, simulate its possession proof using (b), and, knowing the pool scalar x_j , form $(Y, m_bG + x_jY + Z)$; independent uniform Z hides the message even given all committee shares. No SNARK simulation or pool independence is needed for this direction. For the organizer-adversary direction, adding its known term $\mathbf{sk}_{\text{org}}C_1$ transports the automatic experiment. Appendix A.1 details both interfaces.

What the gate does and does not promise. The reveal makes all admitted ciphertexts eligible under one application-wide gate, subject to the decryption window, and with fewer than t corrupt members honest nodes release no partial before that gate opens. Nevertheless, t colluding members defeat an automatic application’s embargo, an organizer with t members decrypts locked ciphertexts off chain without revealing, and a closing deadline cannot revoke published partials. Eligibility is neither simultaneous delivery nor fairness; an authorized submitter still chooses which ciphertexts or aggregates are admitted; and an organizer-secret leak permits unauthorized reveal, but does not itself bypass a still-closed window. Robustness requires at least t available share holders and an open gate: decryption fails with fewer. The chosen epoch tolerates $n - t$ unavailable members; parameter floors alone enforce no positive margin.

7 Implementation and Evaluation

The implementation consists of the node and the four circuits in Go on gnark v0.16.3 [8], the three contracts in Solidity 0.8.28 (via-IR, optimizer runs 1, Cancun rules) and a TypeScript client. Every figure below is for the fixed capacities $N_{\max} = 32$, $K_{\max} = 8$ with the hint-free scalar multiplication of §5. The Groth16 artifacts come from a single-party setup and are pinned by hash in the node, which loads nothing else; a production deployment must replace them by a multi-party ceremony, whose published transcript gives the usual guarantee that soundness holds if one participant destroyed its randomness. Constraint counts and gas do not depend on the ceremony.

Methodology. Constraint counts are those of the compiled R1CS. Proving times are wall-clock per proof, mean of five, with gnark parallelising over the 32 logical threads of an idle AMD Ryzen 9 9950X3D (64 GiB); they include witness solving but not loading the proving key, which is on the order of a gigabyte for the contribution circuit and is loaded once per process. Peak resident memory was not measured. Gas is `gasUsed` from the receipts of a local development chain (Anvil, Cancun rules) running the real verifier contracts of these artifacts, so pairing check, calldata and intrinsic cost are all included; it was measured at $n = 4$, $t = 3$ and $n = 32$, $t = 22$, and over a sweep $n = 4, 8, \dots, 32$ that gives the slopes of Table 4, with the epoch windows widened so that 32 sequential proofs fit. Latency, partial counts and behaviour under delayed or offline members were not measured on this build.

Table 3. Circuit size and proving time at $N_{\max} = 32$, $K_{\max} = 8$. Proving: mean of five proofs, 32 threads, witness solving included, key loading excluded. Inputs: public inputs of the verifier. Transcript: BRLC-committed calldata words; a partial carries its 15 inputs and a 5-word Merkle path instead.

Circuit	Constraints	Inputs	Transcript	Proving time
Contribution	3 060 692	8	928	1 574 ms
Pool-key activation	187 495	8	192	181 ms
Partial decryption	29 026	15	—	32 ms
Combine	287 338	9	102	213 ms

Table 4. Transaction gas (10^3) against the real verifiers on a local development chain (receipts’ `gasUsed`, $N_{\max} = 32$, $K_{\max} = 8$). Slope: gas per additional committee member over the sweep $n = 4, 8, \dots, 32$ (per additional partial for the combine); the other calls stay within 0.1%. Registration is given for the locked / automatic mode. One-off variants: the `createEpoch` resolving the first seed costs 150k, the `claimSlot` resolving an epoch’s seed 205k.

Call	$n=4, t=3$	$n=32, t=22$	Slope	Paid by
<code>createEpoch</code>	133	133	—	creator / epoch
<code>claimSlot</code>	139	120	avg.	member
<code>submitContribution</code>	702	944	+8.7	member
<code>finalizeEpoch</code>	35	35	—	member / epoch
<code>activatePoolKey</code>	508	809	+10.7	member / key
<code>registerApplication</code>	453 / 234	456 / 234	—	organizer / application
<code>revealOrganizerSecret</code>	223	228	—	organizer / locked app.
<code>submitCiphertext</code>	103	103	—	submitter / ciphertext
<code>submitPartialDecryption</code>	402	402	—	t members / ciphertext
<code>combineDecryption</code>	411	484	+3.9	member / ciphertext

Results. In Table 3 the contribution circuit scales with $K_{\max}N_{\max}t$ through its $K_{\max}n$ Horner evaluations, the activation with $N_{\max}t$ plus $2N_{\max} + K_{\max}$ Poseidon absorptions per contributor, the partial is independent of $N_{\max}$, and the combine is dominated by the Vandermonde check’s $N_{\max}$ fixed-base and $N_{\max}^2$ six-bit multiplications. A node thus spends about 1.6 s of CPU per epoch on its contribution, 0.18 s per key it activates, 32 ms per partial and 0.21 s per combine. In Table 4 every proof-carrying call pays about 250k gas for the Groth16 verification itself [10]; the remainder is calldata, of which the contribution’s 29 KiB transcript is the largest item. Transcript lengths are fixed by $N_{\max}$, but absent members’ slots are zero bytes, priced 4 instead of 16 gas, and activation and combine read one storage word per present member, so a contribution grows by about 8.7k gas per member, an activation by 10.7k and a combine by 3.9k per partial, while `claimSlot`’s average falls as the seed-resolving first claim is spread over more members. The verifier’s fixed work is therefore one pairing check with eight public inputs, not a constant-cost transaction: a contribution rises from 702k to 944k gas across the measured committee sizes.

End-to-end cost. Adding the rows of Table 4, an epoch that activates all eight keys—creation, n claims, n contributions, finalization and eight activations—costs 7.60 M gas at $n = 4$ and 40.69 M at $n = 32$, i.e. 0.95 M and 5.09 M per key, the n contributions being 37% and 74% of it; keys are activated on demand, so an unused key costs nothing until activated, but the contributions are sunk. Decrypting one ciphertext (submission, t partials, one combine) costs 1.72 M at $t = 3$ and 9.44 M at $t = 22$, the partials dominating; these totals exclude application registration, the one 0.22 M reveal of a locked application, operator registration and deployment. A single-key ($K_{\max} = 1$) build of the same circuits with the same toolchain and checks was not measured, so no amortization ratio is claimed.

Artifact. The implementation, circuits, contracts and test vectors are available as anonymized supplementary material, together with the pinned toolchain and circuit identifiers, the benchmark procedure behind the tables, and regression tests for the transcript binding of §5.1 and the interpolation check. Its instructions distinguish the single-party experimental setup from the ceremony a production deployment requires.

8 Limitations and Conclusion

Committee selection and liveness. The lottery is eligibility filtering over a snapshot: the seed’s proposer can bias it, slots go to the fastest claimants, and finalization has no deadline; a rushing dealer biases the pool keys by selecting which of its dealings enter QUAL (§6). Robustness depends on the chosen epoch’s $n - t$, which the floors do not bound away from zero, and re-registration protects only future encryptions.

Pool exhaustion and epoch cost. An epoch serves at most $K_{\max}$ applications, and further registrations revert until the next is LIVE; since `createEpoch` may run early once at most one key is unclaimed, $K_{\max} - 1$ permissionless registrations force a new epoch paid for by the committee, and every admitted ciphertext costs it t partials; there is no registration fee or admission list.

Organizer key. sk_{org} is a single point of failure: losing it makes a locked application undecryptable, reusing it lets one reveal open every application it backs, and leaking it permits an unauthorized but visible reveal, never a quiet or early decryption. The gate binds only the contract and honest nodes, not t colluding members or a colluding organizer, and published partials are irrevocable.

Submission and malleability. Without a proof of knowledge, admitted submitters can post $(C_1, C_2 + m'G)$ derived from another ciphertext of the same application; what enters an application is its own responsibility. Identifiers are first come, first served, so clients must verify registrant and key on chain. An out-of-range plaintext costs each combiner a failed discrete-logarithm search; tainting bounds this per registration or submitter, not against fresh addresses.

Trusted computing base. The proving keys come from a single-party setup—pinning hashes is not a ceremony: the combine circuit’s waste permits false plaintexts, the contribution or activation waste a chosen P_j —and the proof library’s gadgets are part of the trusted base. The analysis covers static corruption only [9], and nothing here is post-quantum. Future work: admission economics, beacon-based selection [34,18], proactive refresh [29].

Conclusion. Proof-carrying dealings give an EVM contract a publicly verifiable DKG: one fixed-size verification per contribution, no dispute phase. A pool of independent keys dealt in the same proof, each activated by a small rebinding proof, isolates the applications one committee serves; an optional organizer key adds a one-shot, application-wide release.

References

1. Abdalla, M., Bellare, M., Rogaway, P.: Dhies: An encryption scheme based on the diffie–hellman problem. Tech. rep., Cryptology ePrint / Full version on the authors’ webpages (Sep 2001), <https://web.cs.ucdavis.edu/~rogaway/papers/dhies.pdf>, full version dated September 18, 2001
2. Abraham, I., Bacho, R., Loss, J., Stern, G.: Nearly quadratic asynchronous distributed key generation. IACR ePrint 2025/006 (2025)
3. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G.: Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In: Advances in Cryptology – CRYPTO 2023. pp. 39–70. Springer (2023)
4. Bacho, R., Kavousi, A.: SoK: Dlog-based distributed key generation. In: IEEE Symposium on Security and Privacy (S&P). pp. 614–632 (2025)
5. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898 (2022)
6. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: 1st ACM Conference on Computer and Communications Security (CCS). pp. 62–73 (1993)
7. Benhamouda, F., Halevi, S., Krawczyk, H., Ma, Y., Rabin, T.: SPRINT: High-throughput robust distributed Schnorr signatures. In: Advances in Cryptology – EUROCRYPT 2024. LNCS, Springer (2024), iACR ePrint 2023/427
8. Botrel, G., Piellard, T., Housni, Y.E., Kubjas, I., Tabaie, A.: ConsenSys/gnark: a fast zk-SNARK library that offers a high-level API to design circuits. Software, <https://github.com/ConsenSys/gnark> (2023). <https://doi.org/10.5281/zenodo.5819104>
9. Canetti, R., Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Adaptive security for threshold cryptosystems. In: Advances in Cryptology – CRYPTO ’99. LNCS, vol. 1666, pp. 98–115. Springer (1999)
10. Cardozo, A.S., Williamson, Z.: EIP-1108: Reduce alt_bn128 precompile gas costs. Ethereum Improvement Proposals (2018), <https://eips.ethereum.org/EIPS/eip-1108>
11. Cascudo, I., David, B.: SCRAPE: Scalable randomness attested by public entities. In: Applied Cryptography and Network Security – ACNS 2017. LNCS, vol. 10355, pp. 537–556. Springer (2017)

12. Cascudo, I., David, B.: ALBATROSS: Publicly attestable batched randomness based on secret sharing. In: *Advances in Cryptology – ASIACRYPT 2020*. LNCS, vol. 12493, pp. 311–341. Springer (2020)
13. Cash, D., Kiltz, E., Shoup, V.: The twin diffie–hellman problem and applications. In: *Advances in Cryptology – EUROCRYPT 2008*. *Lecture Notes in Computer Science*, vol. 4965, pp. 127–145. Springer (2008). https://doi.org/10.1007/978-3-540-78967-3_8
14. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: *Advances in Cryptology – CRYPTO ’92*. LNCS, vol. 740, pp. 89–105. Springer (1993)
15. Das, S., Xiang, Z., Kokoris-Kogias, L., Ren, L.: Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In: *32nd USENIX Security Symposium*. pp. 5359–5376 (2023)
16. Das, S., Yurek, T., Xiang, Z., Miller, A., Kober, L., Gao, L.: Practical asynchronous distributed key generation. In: *2022 IEEE Symposium on Security and Privacy (SP)*. pp. 2518–2534. IEEE (2022)
17. Desmedt, Y., Frankel, Y.: Threshold cryptosystems. In: *Advances in Cryptology – CRYPTO ’89*. LNCS, vol. 435, pp. 307–315. Springer (1990)
18. drand contributors: drand: A distributed randomness beacon daemon. Project website (2024), <https://drand.lol/>, accessed: 2026-04-29
19. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: *28th Annual Symposium on Foundations of Computer Science*. pp. 427–438. IEEE (1987)
20. Feng, H., Tang, Q.: Asymptotically optimal adaptive asynchronous common coin and DKG with silent setup. In: *Advances in Cryptology – CRYPTO 2025*. pp. 647–680. Springer (2025)
21. Fouque, P.A., Stern, J.: One round threshold discrete-log key generation without private channels. In: *Public Key Cryptography – PKC 2001*. LNCS, vol. 1992, pp. 300–316. Springer (2001)
22. Freivalds, R.: Probabilistic machines can use less running time. In: *Information Processing 77, Proceedings of IFIP Congress 77*. pp. 839–842. North-Holland (1977)
23. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: *Advances in Cryptology – EUROCRYPT ’99*. LNCS, vol. 1592, pp. 295–310. Springer (1999), full version: *Journal of Cryptology* 20(1):51–83, 2007
24. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure applications of Pedersen’s distributed key generation protocol. In: *Topics in Cryptology – CT-RSA 2003*. LNCS, vol. 2612, pp. 373–390. Springer (2003)
25. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schafneggler, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: *30th USENIX Security Symposium*. pp. 519–535 (2021)
26. Groth, J.: On the size of pairing-based non-interactive arguments. In: *Advances in Cryptology – EUROCRYPT 2016*. *Lecture Notes in Computer Science*, vol. 9666, pp. 305–326. Springer (2016). https://doi.org/10.1007/978-3-662-49896-5_11
27. Groth, J.: Non-interactive distributed key generation and key resharing. *IACR Cryptology ePrint Archive*, Report 2021/339 (2021), <https://eprint.iacr.org/2021/339>
28. Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Aggregatable distributed key generation. In: *Advances in Cryptology – EUROCRYPT 2021*. LNCS, vol. 12696, pp. 147–176. Springer (2021)

29. Herzberg, A., Jarecki, S., Krawczyk, H., Yung, M.: Proactive secret sharing or: How to cope with perpetual leakage. In: *Advances in Cryptology — CRYPTO '95. Lecture Notes in Computer Science*, vol. 963, pp. 339–352. Springer (1995)
30. Hisil, H., Wong, K.K.H., Carter, G., Dawson, E.: Twisted Edwards curves revisited. In: *Advances in Cryptology – ASIACRYPT 2008. LNCS*, vol. 5350, pp. 326–343. Springer (2008)
31. Kate, A., Goldberg, I.: Distributed key generation for the internet. In: *29th IEEE International Conference on Distributed Computing Systems (ICDCS)*. pp. 119–128 (2009)
32. Kate, A., Mangipudi, E.V., Mukherjee, P., Saleem, H., Thyagarajan, S.A.K.: Non-interactive VSS using class groups and application to DKG. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024)*. ACM (2024), *cryptology ePrint Archive*, Paper 2023/451
33. Kokoris-Kogias, E., Malkhi, D., Spiegelman, A.: Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1751–1767 (2020)
34. Micali, S., Rabin, M.O., Vadhan, S.P.: Verifiable random functions. In: *40th Annual Symposium on Foundations of Computer Science (FOCS 1999)*. pp. 120–130. IEEE Computer Society (1999)
35. Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: *Advances in Cryptology—EUROCRYPT'91*. pp. 522–526. Springer (1991)
36. Schindler, P., Judmayer, A., Stifter, N., Weippl, E.: ETHDKG: Distributed key generation with Ethereum smart contracts. *Cryptology ePrint Archive*, Report 2019/985 (2019)
37. Schoenmakers, B.: A simple publicly verifiable secret sharing scheme and its application to electronic voting. In: *Advances in Cryptology – CRYPTO '99. LNCS*, vol. 1666, pp. 148–164. Springer (1999)
38. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM* **27**(4), 701–717 (1980)
39. Shamir, A.: How to share a secret. *Communications of the ACM* **22**(11), 612–613 (1979). <https://doi.org/10.1145/359168.359176>
40. Shanks, D.: Class number, a theory of factorization, and genera. In: *Proceedings of Symposia in Pure Mathematics*. vol. 20, pp. 415–440. AMS (1971)
41. Shoup, V., Gennaro, R.: Securing threshold cryptosystems against chosen ciphertext attack. *Journal of Cryptology* **15**(2), 75–96 (2002), preliminary version in *EUROCRYPT '98*
42. Sober, M., Kobelt, M., Scaffino, G., Kaaser, D., Schulte, S.: Distributed key generation with smart contracts using zk-SNARKs. In: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC)*. pp. 229–238. ACM (2023). <https://doi.org/10.1145/3555776.3577677>, best Paper Award, Distributed Systems Track
43. Stadler, M.: Publicly verifiable secret sharing. In: *Advances in Cryptology – EUROCRYPT '96. LNCS*, vol. 1070, pp. 190–199. Springer (1996)
44. Team, S.N.: Shutter: Distributed key generation with applications to front-running protection. *Cryptology ePrint Archive*, Report 2024/1981 (2024), deployed on Gnosis Chain since October 2022
45. Wuille, P.: BIP-0032: Hierarchical deterministic wallets. *Bitcoin Improvement Proposal* (2012), <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>, accessed: 2026-04-29

46. Zhang, H., Duan, S., et al.: Practical asynchronous distributed key generation: Improved efficiency, weaker assumption, and standard model. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 568–581 (2023)

A Reduction details

A.1 Locked-application bookkeeping

First finish the epoch using real circuit proofs. When $n - f \geq t$, Lemma 2 lets the reduction recover each adversarial polynomial from t honest recipients; otherwise it invokes assumption (d)’s knowledge extractor on the accepted contributions, adding extractor running time and failure probability ϵ_{ks} —not a straight-line reconstruction claim. Extraction precedes organizer registration, so it needs no soundness after simulated SNARK proofs.

To simulate a possession proof for X , sample $h \leftarrow \{0, 1\}^{256}$ and $z \leftarrow \mathbb{F}_q$, and set $c = h \bmod q$, $T = zG - cX$, $H_K(\text{ctx}_{\text{org}}, X, T) := h$, where the context holds the registration domain, epoch, and application identifier. Programming the *full* output preserves the actual reduced-challenge distribution; as T is uniform, an existing-query conflict costs at most Q_K/q per registration, so switching at most Q_A honest registrations costs $Q_A Q_K/q$.

Guess the challenge registration and embed the DDH element there; other organizer secrets remain known and their reveals are answered normally, while the target secret is neither revealed nor supplied through an organizer-DH oracle, although all committee partials could be disclosed. The challenge uses the known pool scalar as in Theorem 2; unsuccessful guesses return random bits, yielding the factor Q_A .

For the organizer-adversary direction, the experiment supplies the target’s correctly generated organizer secret as auxiliary input; ciphertext translation adds its known multiple of C_1 , and all target honest partials remain gated. This corollary does not analyze arbitrary malicious target-key generation merely from a successful possession verification.

A.2 Automatic-application hybrids and composition

For dealer d , recipient u , and key j , the actual mask structure is $S_{d,u} = r_{d,u} \text{pub}_u$, $M_{d,u,j} = \text{Hp}(\text{dom}_{\text{mask}}, \text{eid}, 2^{16}d + u, j)$, and $\sigma_{d,u,j} = f_d^{(j)}(u) + \text{Hp}(M_{d,u,j}, S_{d,u}.x, S_{d,u}.y) \pmod{q}$. Index bounds make the packing injective, and, except for hash collisions already charged to ϵ_{bind} , different keys have different outer-mask inputs despite sharing $S_{d,u}$.

After simulating honest contribution proofs, a per-row hashed-ElGamal argument [1] first replaces the row’s DH secret by an independent random group point; unless that point is queried, its K_{max} hash outputs are independent field elements, so knowledge of other keys’ shares and masks does not expose the target mask. Outside the aggregate extraction and collision bad events, a sufficient per-row bound is $\epsilon_{\text{HE}} \leq \text{Adv}_{\mathcal{G}}^{\text{ddh}}(\mathcal{B}_{\text{row}}) + (Q_P + Q_K)/q + K_{\text{max}}\beta$ with

$\beta = \eta(q - \eta)/(pq)$ and $\eta = p - 7q$, where Q_P counts relevant outside queries to the mask oracle, Q_K/q covers simulation of the embedded recipient's registration proof, and β is exactly the statistical distance between $U(\mathbb{F}_p) \bmod q$ and $U(\mathbb{F}_q)$: η residues have eight preimages and the others seven.

There is an important oracle restriction: the row reduction may not know its embedded recipient's secret. Assumption (f)'s online extractor supplies accepted adversarial contribution witnesses, hence their polynomials, so honest virtual states and other-key partials can still be generated; honest encrypted rows are *not* exposed through a decryption oracle, and subsequent hybrid states retain the original mathematical shares rather than decrypting placeholders. Only honest contributor contexts receive simulated proofs, and the contract's authenticated contributor index and one-contribution rule prevent an adversarial dealing from reusing such a context, so every newly accepted corrupt dealing falls within assumption (f)'s freshness condition. Ordinary zero knowledge alone supplies neither this guarantee nor the required extraction interface.

For the final DDH embedding, choose $t - 1$ nonzero interpolation nodes containing all corrupt positions, assign them independent known evaluations v_z , let L_0, L_z be the basis polynomials for those nodes together with zero, and set $F(T)G = L_0(T)X + \sum_z L_z(T)v_zG$, a uniformly distributed degree-below- t polynomial in the ideal sampling experiment with known corrupt evaluations; every other honest polynomial is sampled normally. All recipient encryption secrets are now known, so t honest decryptions recover each adversarial polynomial without extraction. Activation is genuinely provable from commitment points and their digests: its transcript digest covers the masked $6N_{\max}$ words together with `eid`, j , occupies input index 5, and enters the BRLC anchor, and no hidden polynomial scalar is its witness.

Guess the target slot and the lowest-index honest dealer in `QUAL`, testing both guesses before using the adversary's output; this gives $L \leq K_{\max}(n - f)$ without assuming a dealer is guaranteed acceptance. The encryption hybrids run over all R rows before this guessing reduction, so their losses are additive outside L . Adaptive offsets may bias the final keys, but the recovered scalar offset still permits the final DDH translation. All reductions include their proving and extraction overhead.

A.3 Canonical share-mask reduction

Write $p = 7q + \eta$, where $0 < \eta < q$. For a canonical raw hash value $h \in [0, p)$, the circuit enforces $h = \nu q + \mu \pmod{p}$ with $0 \leq \nu \leq 7$, $0 \leq \mu < q$, and $\nu = 7 \implies \mu < \eta$. These bounds imply $\nu q + \mu < p$ over the integers, so the field equality is an integer equality, giving the unique canonical remainder $h \bmod q$; a remainder bound alone would not suffice. Masked-share addition also range-checks the share, mask, and result below q , and restricts its carry to $\{0, 1\}$; both sides of that equality are below $2q < p$, excluding native-field wraparound there as well.